

Systeme d'alarme

pour maison connectée

Team EEE

Evan Galli, Élian Delmas, Eliot Menoret

Introduction

Nano ordinateur:

- capable de tourner avec des OS complexes
- interactions de haut niveau
- pas de temps réel

Microcontrôleurs:

- Réactifs
- Capable de temps réel
- Limités en mémoire et traitement de l'information

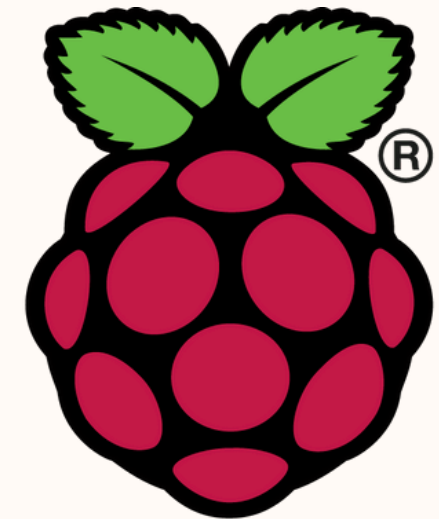
Introduction

Nano ordinateur:

- capable de tourner avec des OS complexes
- interactions de haut niveau
- pas de temps réel

Microcontrôleurs:

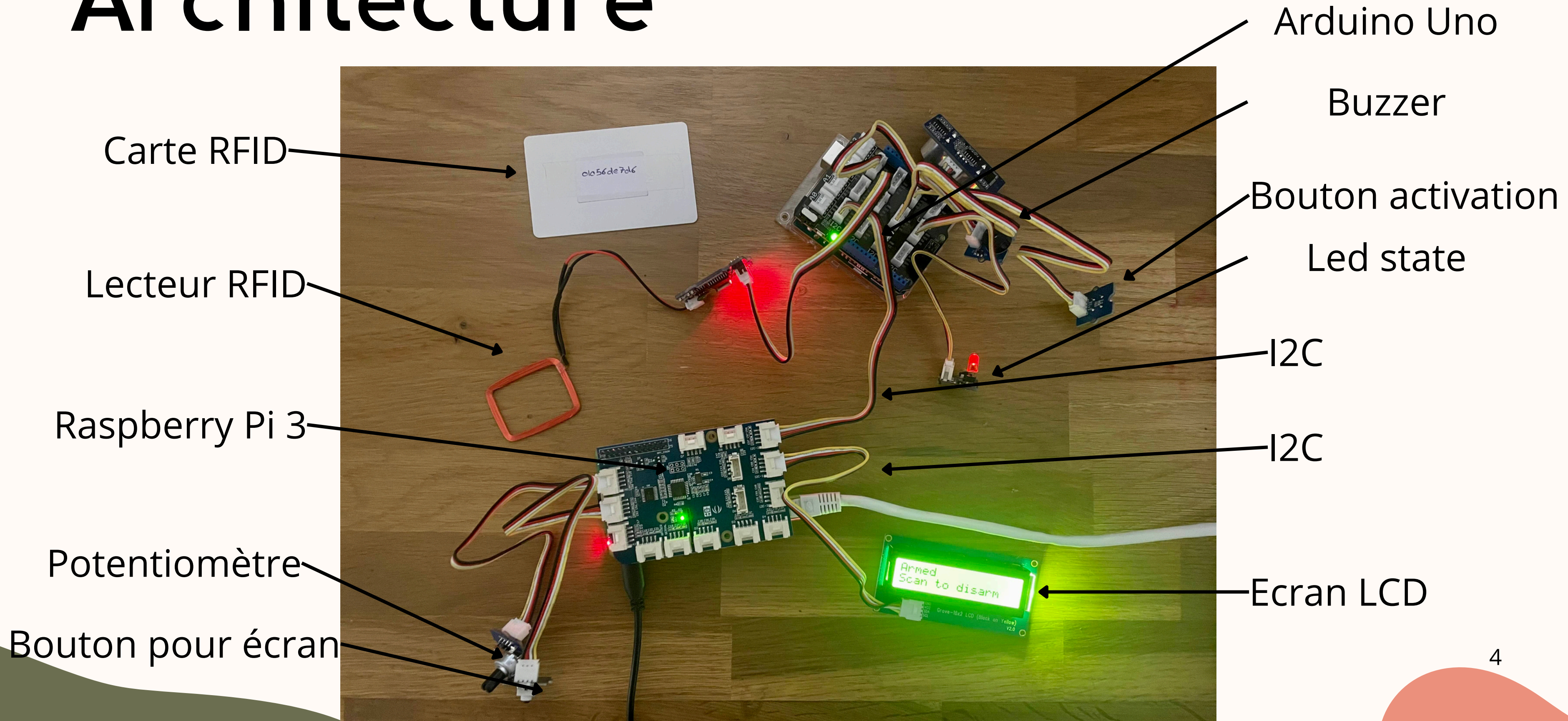
- Réactifs
- Capable de temps réel
- Limités en mémoire et traitement de l'information



Raspberry Pi



Architecture



Arduino Uno ●

FreeRTOS :

- ordonnancement préemptif (assure la disponibilité du bus I2C pour la communication)
- acquisition temps réel
- optimisation des ressources CPU grâce aux Event Groups

Architecture des Tâches :

- vUltrasonicTask : Surveillance continue anti-intrusion
- vReadRfid : Lecture régulière des info du lecteur RFID
- vButtonTask : Gestion de l'activation de l'alarme
- vBlinkTask : Clignotement visuel LED et activation sonore du buzzer

Protocole I2C Arduino-Pi

Adresse	Nom	Description
0x00	REG_STATUS	État du système (Désarmé, Armé, Triggered). Contrôle la machine à états.
0x01	REG_EVENTS	Drapeaux d'événements (Bitmask). Indique quels capteurs ont été activés
0x02..0x07	REG_RFID	Contient l'UID du dernier badge RFID détecté (6 octets)
0x08	REG_ULTRASONIC_DISTANCE	Seuil de distance pour la détection de mouvement.

Fiabilisation de l'I2C ●

Mise en place checksum:

- Checksum: $\sum$ (octets) pour valider chaque paquet.
- Écriture : [REG] + [CHECKSUM] + [DATA]
- Lecture : [DATA] + [CHECKSUM]

Mutex sur le bus I2C:

- garantit l'intégrité des lectures et écritures des registres

Machine à état

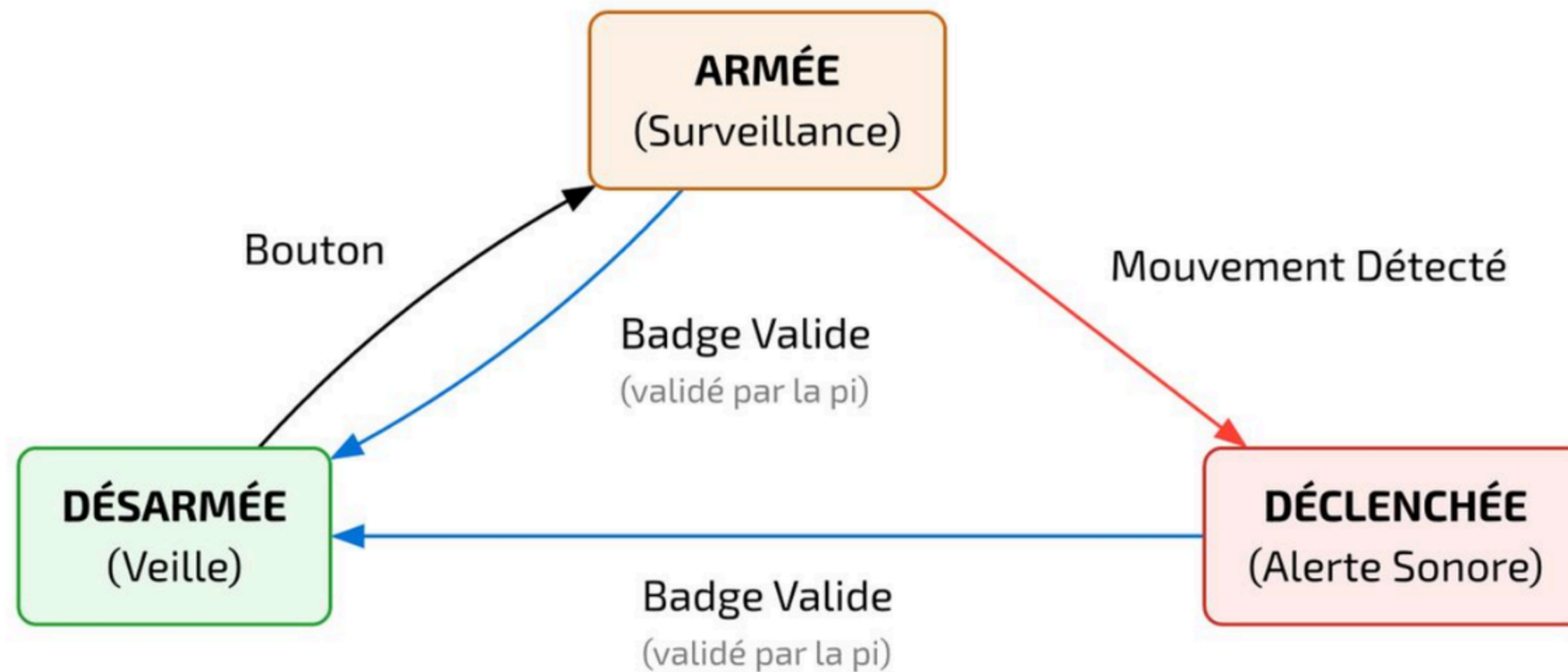


Fig. 3. – Machine à états finis de l'alarme

Raspberry Pi ● (Raspberry Pi OS Lite)

Fonctionnalités

2 modes d'exécution :

- mode headless (défaut)
- mode TUI (--tui)

Gestion des badges :

- Base de donnée SQLite
- UID, état d'activation, dernière utilisation, expiration

Interface LCD :

- Contrôles via bouton et potentiomètre
- Statut actuel directement visible

Configurable :

- Ajout/suppression de badge
- Ajustement distance de déclenchement

Problèmes rencontrés

Raspberry Pi:

- Race condition pour les lectures I2C → réécriture le code des routines d'interruption
- Données corrompues sur le bus I2C → mise en place de checksum
- Shield GrovePi incompatible → remplacement par un shield GrovePi+
- Lecture impossible sur les ports digitaux → utilisation port analogique + seuillage logiciel

Arduino:

- Mémoire disponible très limitée → Réduction de la heap FreeRTOS
- Débogage plus complexe dû à l'aspect microcontrôleur

Évolutions possibles

1. Synchroniser les badges valides dans l'EEPROM de l'Arduino :
 - microcontrôleur peut valider un désarmement
 - continuité en cas de rupture I2C
2. Sécuriser l'accès au menu de configuration en demandant le scan d'un badge autorisé
3. Ajouter un état « pré-alarme » (délai configurable) pour atteindre le lecteur RFID avant l'activation du buzzer
4. Découpler la TUI du programme daemon
5. Donner accès à plus de fonctionnalités sur l'écran LCD

Démo

Merci pour votre écoute !